OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Communication Patterns – Forwarder-Receiver Pattern

Archana A
Department of Computer Applications

2. Forwarder-Receiver Pattern

- The Forward-Receiver design pattern provides **transparent inter-process communication (IPC)** for software systems with a peer-to-peer interaction model.
- It introduces **forwarder and receivers** to decouple peers from the underlying communication mechanisms.

- This design pattern provides **Encapsulation.**

- **Context:**

  Peer-to-peer communication

Forwarder-Receiver Pattern

Problem:

- When distributed applications are built using available low level mechanisms for inter-process communication, **it introduces dependencies on the underlying operating system** and network protocols.

  - It **restricts portability**
  - It **constrains** the system's capability to support heterogeneous environments.
  - It makes it **hard to change the IPC mechanism later**.

Solution:

- The details of the underlying IPC mechanism for **sending or receiving messages are hidden** from the peers by **encapsulating all such functionalities into separate components.**
- Eg: **mapping names to physical locations, establishment of communication channels, and marshaling and un-marshaling of messages**.

Structure:

- The Forwarder-Receiver design pattern consists of three kinds of components:
  - Forwarders
  - Receivers
  - Peers

Forwarder-Receiver Pattern

Peer components

- Responsible for application tasks.
- They need to communicate with other peers (located in a different process or on a different machine) to carry out their tasks.
- It uses a **forwarder to send messages** to other peers and a **receiver to receive messages** from other peers, such messages are either **requests or responses.**

Class Peer	***Collaborators*** • Forwarder • Receiver
Responsibility • Provides application services. • Communicates with other peers.	

Forwarder-Receiver Pattern

Forwarder component

- Send messages across process boundaries.
- Provides a general interface that is an abstraction of a particular IPC mechanism
- It also contains a **mapping from names to physical addresses.**

Class Forwarder	***Collaborators*** • Receiver
Responsibility • Provides a general interface for sending messages. • Marshals and delivers messages to remote receivers. • Maps names to physical addresses.	

Receiver component

- Responsible for receiving messages.
- Provides a general interface that is an abstraction of particular IPC mechanism.
- Includes functionality for **receiving and unmarshaling** messages.

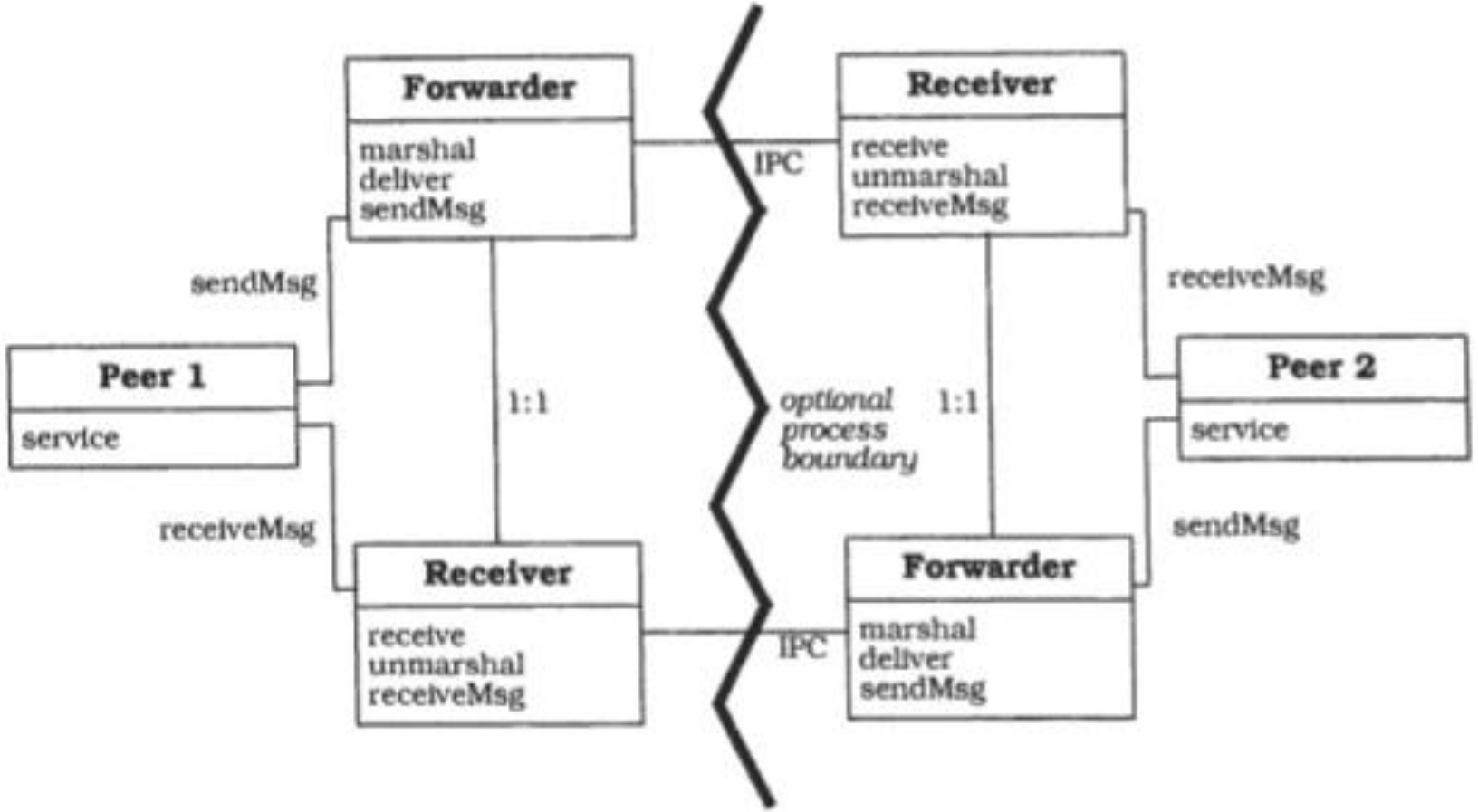
.

Class Receiver	***Collaborators*** • Forwarder
Responsibility • Provides a general interface for receiving messages. • Receives and unmarshals messages from remote forwarders.	

General Structure:

To send a message to a remote peer, the peer invokes the method sendMsg of its forwarder, passing the message as an argument

.



- This method must convert messages to a format that underlying **IPC mechanism** understands. For this purpose it calls ***marshal.sendMsg*** uses *deliver* to transmit that IPC message data to a remote receiver.

- When a **peer wants to receive a message** from a remote peer. It invokes the **receiveMsg** method of its receiver, and the message is returned. ***receiveMsg*** **invokes** ***receive***, which uses the functionality of the underlying IPC mechanisms to convert IPC messages to a format that the peer understands.

.

Variants:

- Forwarder-Receiver without name-to-address mapping
- Peers tell their forwarder the physical location of the recipient.
- **Advantage:** better performance.
- **Disadvantage:** significantly decrease the ability to change the IPC mechanism.

Known Uses:

- the material flow control software for flexible manufacturing that was developed as a part of REBOOT project uses F-R structure to facilitate an efficient IPC.
- An ATM-P switching system uses F-R design pattern to implement the IPC between statically distributed components

Consequences:

➢ **Benefits:**

- Efficient inter-process communication.
- Encapsulation of IPC facilities.

➢ **Liability:**

- No support for flexible reconfiguration of components.
  - Forwarder-Receiver systems are hard to adapt if the distribution of peer may change at run-time.

For example

1. Remote Control and Devices:

Forwarder: The remote control acts as the forwarder. It receives user input (button presses) and doesn't perform any actions itself.

Receiver: The actual device being controlled (TV, stereo, etc.) acts as the receiver. It receives the signal from the forwarder (via infrared or radio waves) and performs the corresponding action (change channel, adjust volume, etc.).

2. Payment Gateways and Online Stores:

Forwarder: An online store acts as the forwarder. It collects customer payment information during checkout.

Receiver: A secure payment gateway acts as the receiver. The online store forwards the payment details to the gateway, which securely processes the transaction with the customer's bank and informs the store of the outcome.

3. Messaging Apps and Push Notifications:

Forwarder: A messaging app on a smartphone acts as the forwarder. It receives incoming messages from the server.

Receiver: The smartphone's operating system acts as the receiver. The app forwards the received message data to the OS. The OS then triggers a push notification, alerting the user about the new message.

THANK YOU

Ms. Archana A
Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392